



Kubernetes

From Containers to Clusters



**I DON'T ALWAYS
DEPLOY ON KUBERNETES**



**BUT WHEN I DO,
IT'S A REAL CLUSTER**

Lets provision a Kubernetes cluster

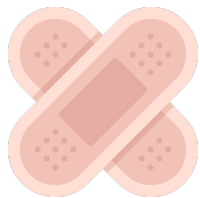
Create a directory `.kube` in your HOME directory

- Windows - `C:\Users\<username>\.kube`
- OSX - `/Users/<username>/.kube`

Copy **config** (kubeconfig) into `.kube` directory

`kubectl cluster-info`

Kubernetes in a Nutshell



Self healing



Configuration management



Scaling



Upgrades and
rollback

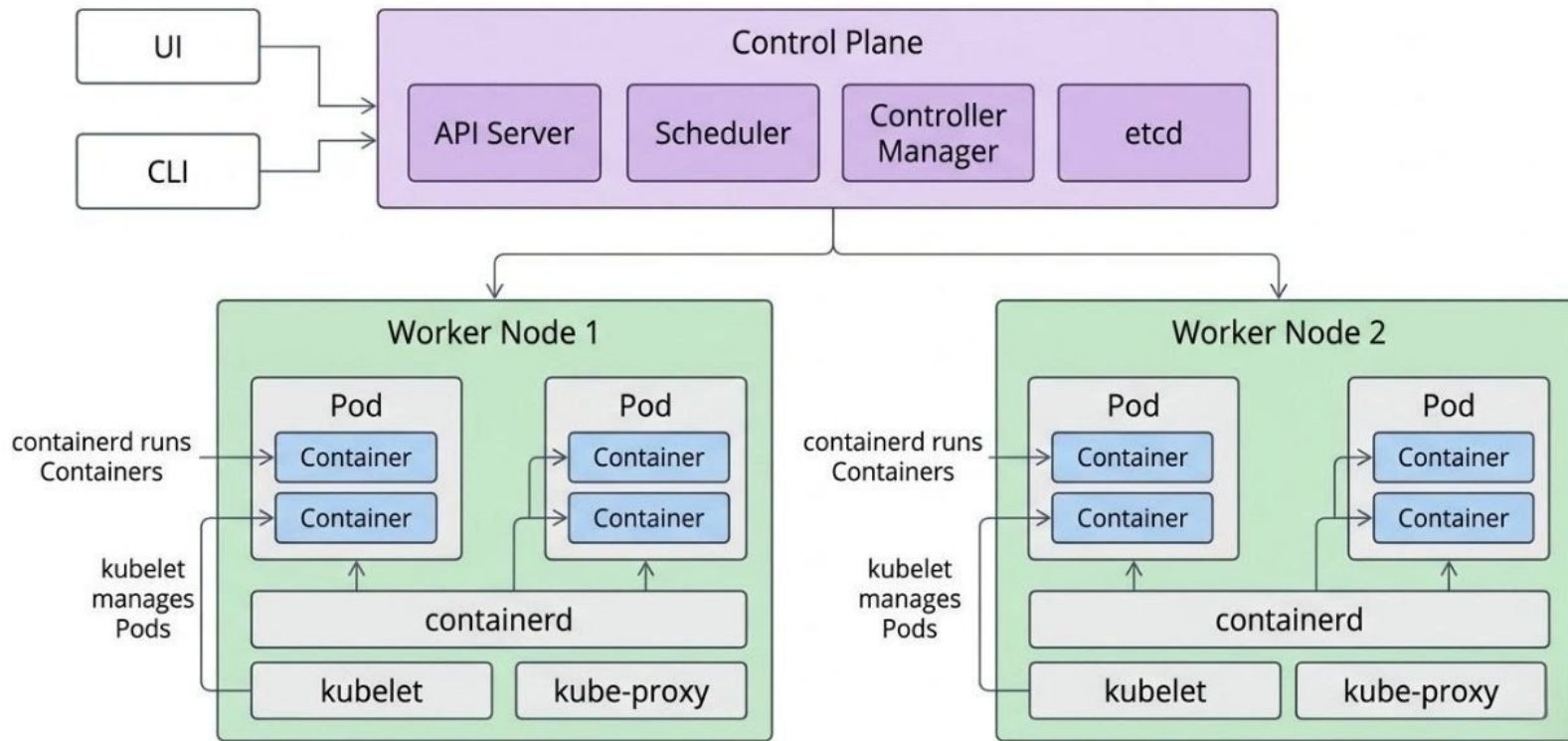


Service
discovery

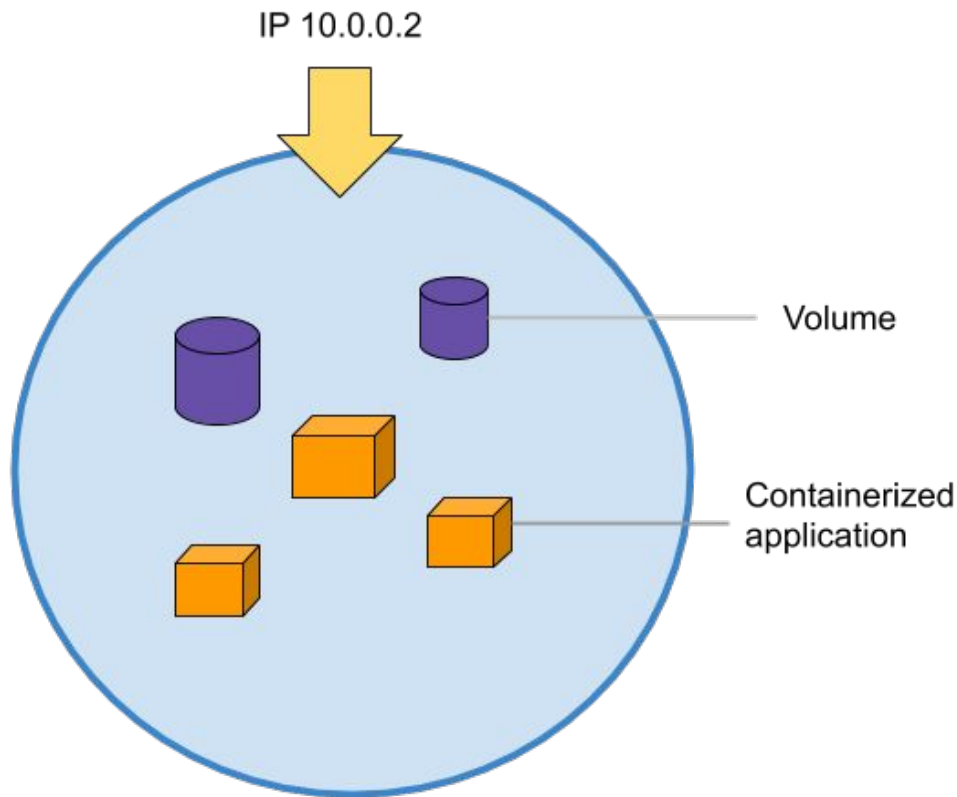


Multi cloud

Architecture



Pod



A Pod is the smallest, most basic deployable unit in Kubernetes.

- **The "Wrapper" for Containers:** A Pod can hold one or more or multiple tightly-coupled containers that need to work together.
- **Shared Identity & Network:** Every container inside a single Pod shares the exact same IP address and port space. They can talk to each other using simply `localhost`.
- **Shared Storage:** Containers in the same Pod can access the same shared storage volumes, making it easy to share data files.
- **Ephemeral by Design:** If a Pod fails, crashes, or its node dies, Kubernetes doesn't try to fix that specific Pod—it simply creates a brand-new identical one to replace it.

Deploy a Pod

Image:

- stefanprodan/podinfo:6.14.1

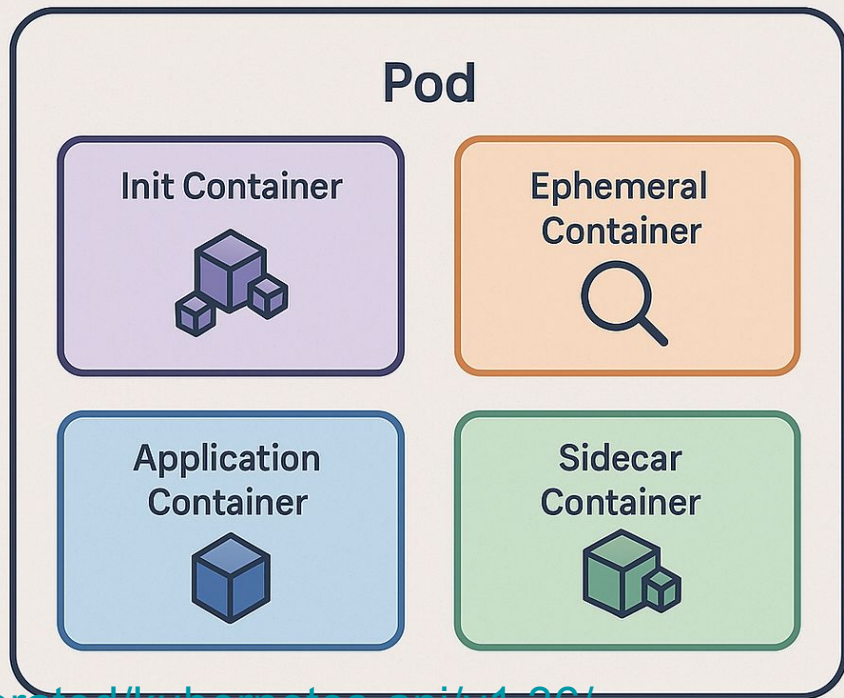
Port:

- 9898

Environment variables

- PODINFO_UI_COLOR (#34577c)
- PODINFO_UI_MESSAGE

Types of Containers in Kubernetes



<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.36/>

<https://bit.ly/3UDMHKy>

<https://github.com/chukmunlee/geekshacking-aug29-2026>

Command Cheat Sheet

kubectl **<command>** **<resource>**/**<name>** **<options>** -n **<namespace>**

Command

- get, describe, delete, create, log, apply, port-forward

Resource

- namespace, pod, deploy, service, gateway, httproute
- all - special name for deploy, service, pod, replicaset

Name

- Resource name, optional
- Will list all resource type if name is missing

Options

- get - -owide
- apply - -f

Namespace

- default namespace if not specified



Deployment

How do I get N identical, stateless pods?

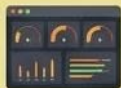
Core Concepts

Stateless

Replicas are Interchangeable

Random Pod Names

Easy Scale & Updates



Grafana



Replicas: 4



gf-abc12



gf-xyz34



gf-pfx18



gf-mar13

Pod Replicas (on any node)



StatefulSet

How do I get N pods with stable identity & storage?

Core Concepts

Stateful

Unique Identities

PersistentVolume per Pod

Ordered Deployment (0, 1,...)



MySQL



Replicas: 4



mysql-0



mysql-1



mysql-2



mysql-3

Stateful Pods



DaemonSet

How do I run one pod per node automatically?

Core Concepts

Node-level Workloads

Every Node gets one

Scales with the Cluster



Fluentd

Node 1



fluentd-66zm

Node Agent

Worker Node

Node 2

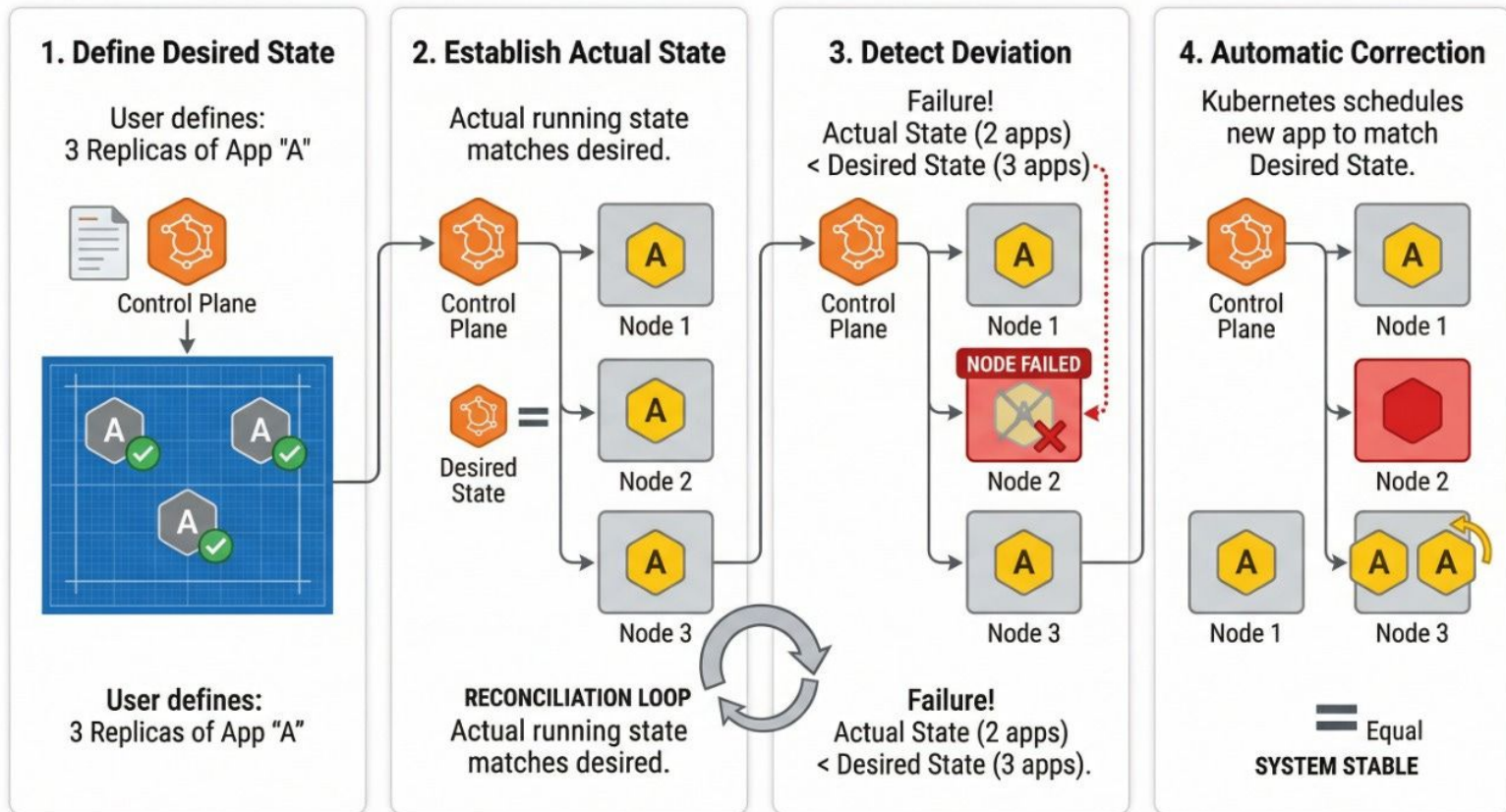


fluentd-f6kv

Node Agent

Worker Node

Deployment - How Does It Work



Create a Deployment

Replicas:

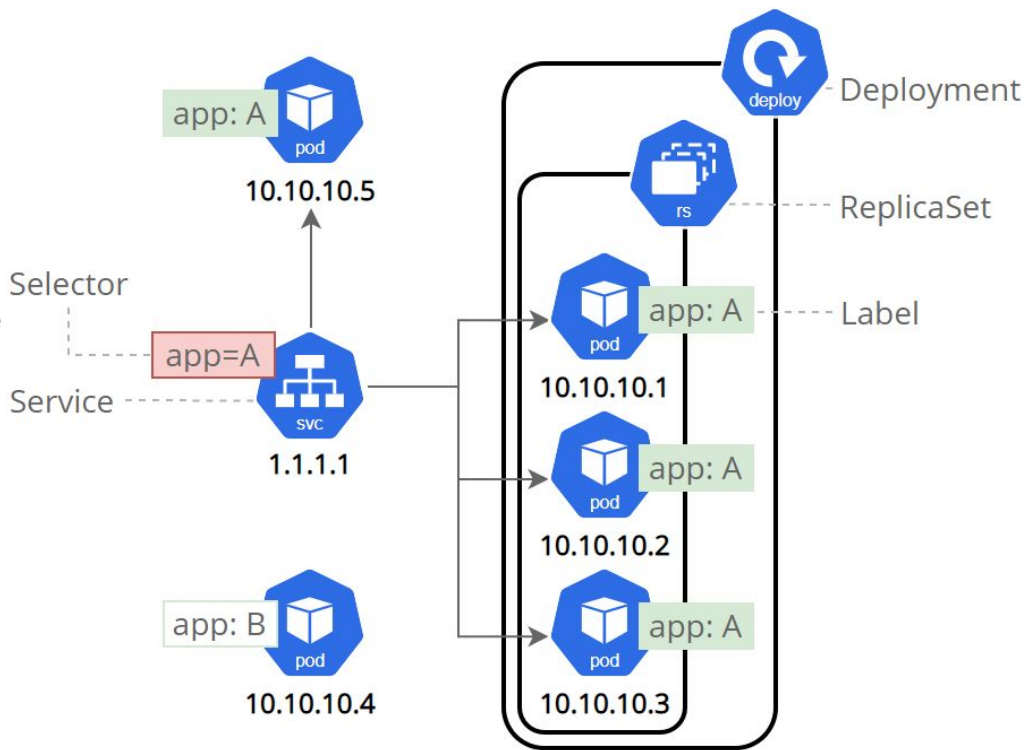
- 3



Service

A Service provides a stable IP and DNS name to reliably routes traffic to Pods

- **Stable Identity:** A Service gives your application a persistent IP address and a stable DNS name within the cluster. Even if every single backend Pod crashes and gets replaced with new IPs, the Service's address remains exactly the same.
- **Built-in Load Balancing:** When traffic hits a Service, it distributes that network traffic evenly across all the healthy Pods that match its label selector.
- **Decoupling:** Services decouple the frontend of your application from the backend. The frontend only needs to talk to the Service name (e.g., `http://backend-api`), unaware of how many Pods are currently running or what their individual IPs are.



Create a Service

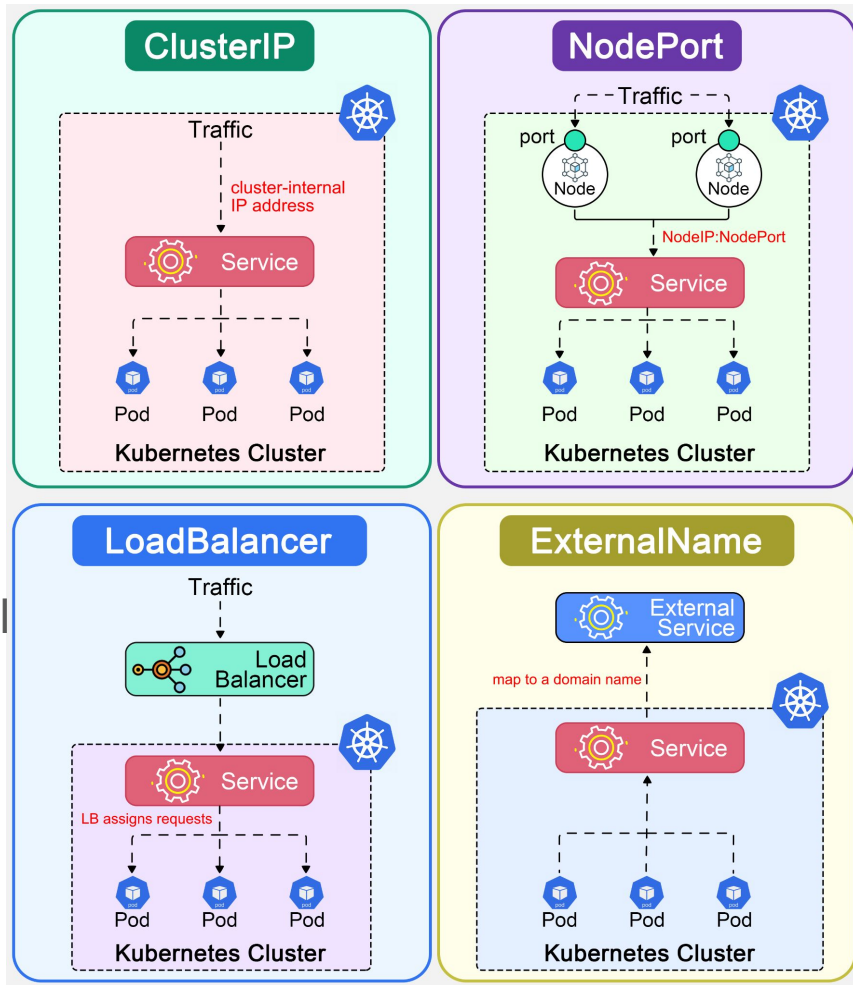
Service port:

- 80

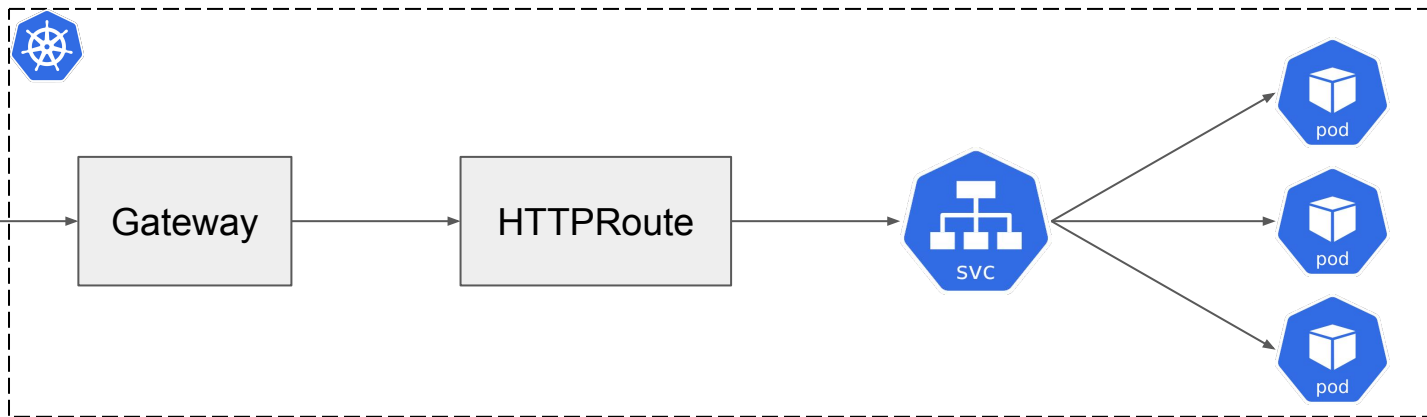
Target port:

- 9898

`<service_name>.<namespace>.svc.cluster.local`



Gateway



Provide a standardized way of routing L4 and L7 traffic into the cluster

- Evolution of and successor to Ingress

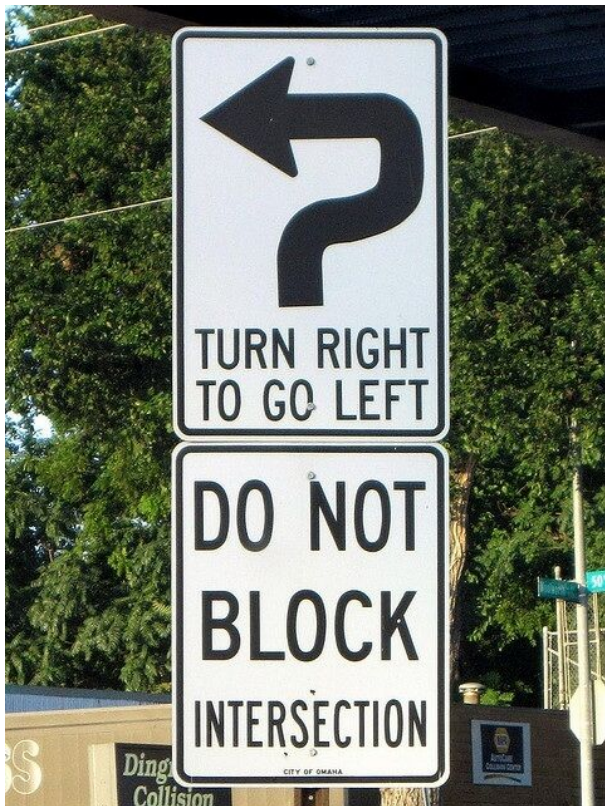
Gateway represents the load balancer at the edge of the cluster

- Identifies how traffic enters the cluster
- Receive traffic on port 443 with this certificate

HTTPRoute resource performs the actual routing

- Receives traffic from Gateway and define where the traffic is forwarded to
- Eg. Forward 90% of the /api/v1 to service A, and the rest for /api/v2 to service B

Deploy and Gateway to Route Traffic to Your Service



Gateway

- Port: 80
- Protocol: HTTP
- Route *.nip.io hosts
- Accessible from all namespaces

HTTPRoute

- Bind to hostname
podinfo.<gateway_ip>.nip.io
- Forward to your service

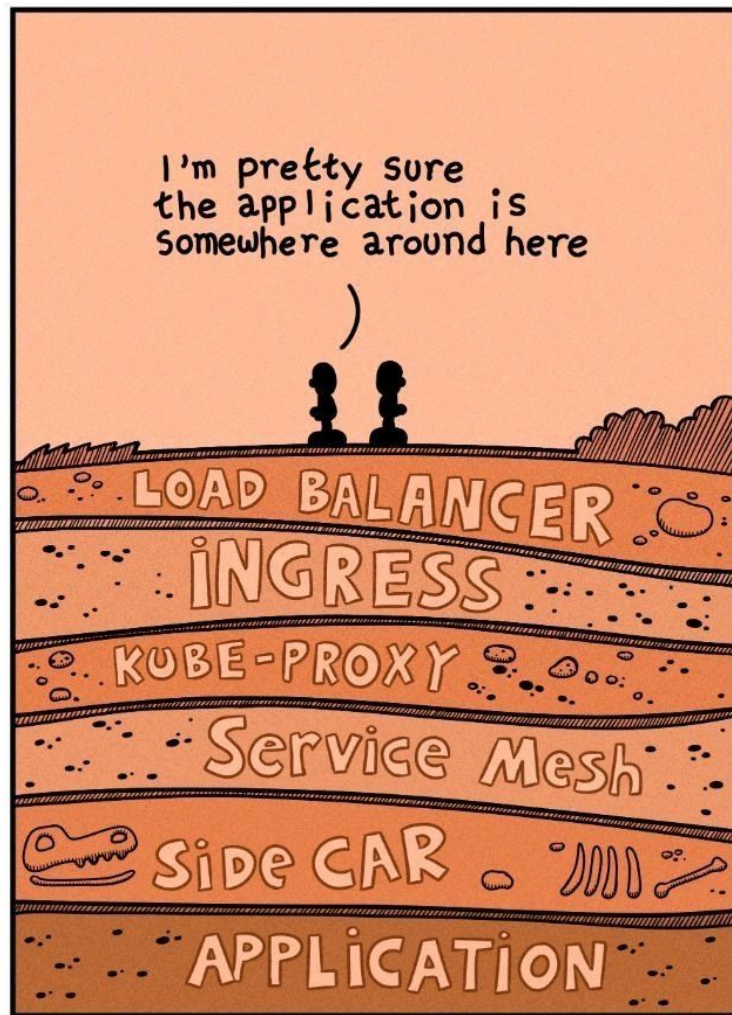
Abstraction

Kubernetes manages complexity by layers

- Every layer hides the operational details of the layer below

Layers

- Ingress and routing (Gateway, HTTPRoute) - handles external ingress and route traffic to service based on paths, headers and weights
- Workload controller (Deployment) - reconcile the desired state with the actual state
- Execution unit (Pod) - host one or more containers which does the actual work



Next Steps



Try it
yourself



Deploy the following

image: chukmunlee/2048:v1

port: 8080

replicas: 3

The Illustrated Children's Guide to Kubernetes

